

Memory Management in Java

Java manages memory automatically using the **JVM (Java Virtual Machine)**. It handles memory allocation and deallocation through a process called **Garbage Collection (GC)**, which removes unused objects to prevent memory leaks.

◆ Two Types of Memory in Java

Java divides its runtime memory into two main areas:

1. Stack Memory (Static Memory Allocation)

🔑 Purpose:

Used for **method execution** and **local variable storage**.

📌 Key Characteristics:

- Stores **primitive data types** (e.g., int, float, char, etc.)
- Stores **references** to objects (not the actual objects themselves).
- Each **thread** has its own stack.
- **Memory is automatically freed** when the method call ends.
- Faster access compared to heap.

⚙️ Example:

```
public class StackExample {  
    public static void main(String[] args) {  
        int a = 10;    // stored in stack  
        int b = 20;    // stored in stack  
        int sum = add(a, b);  
        System.out.println(sum);  
    }  
  
    static int add(int x, int y) {  
        int result = x + y; // result lives in stack until method ends
```

```
    return result;
```

```
    }
```

```
}
```

When add() finishes, all its local variables (x, y, result) are popped from the stack.

2. Heap Memory (Dynamic Memory Allocation)

🔑 Purpose:

Used for **storing objects** and **class instances** at runtime.

🔑 Key Characteristics:

- All **objects and their instance variables** are stored here.
- **Shared** among all threads.
- **Slower access** compared to stack.
- Managed by **Garbage Collector (GC)**, which removes objects with no references.
- Can lead to **OutOfMemoryError** if not managed properly.

⚙️ Example:

```
public class HeapExample {
```

```
    public static void main(String[] args) {
```

```
        Person p1 = new Person("Alice"); // object stored in heap
```

```
        Person p2 = new Person("Bob");   // another object in heap
```

```
    }
```

```
}
```

```
class Person {
```

```
    String name; // stored in heap along with object
```

```
    Person(String name) {
```

```
        this.name = name;
```

}

}

The references p1 and p2 are in the stack, but their corresponding Person objects are stored in the heap.

Garbage Collection (GC) is the **automatic memory management** process in Java performed by the **JVM (Java Virtual Machine)**.

Its main job is to **identify and remove objects that are no longer reachable**, thereby freeing up heap space.

In simple terms — **GC cleans up memory by deleting objects that your program can no longer reach.**

◆ Why Do We Need GC?

Before Java, in languages like C or C++, developers had to manually allocate and free memory using malloc() and free().

This often led to:

- **Memory leaks** (memory not freed)
- **Dangling pointers** (accessing freed memory)
- **Crashes**

GC solves this by:

- **Automatically reclaiming memory** for unused objects.
 - **Reducing memory leaks** and developer burden.
 - Allowing developers to focus on logic instead of memory management.
-

◆ How GC Works (JVM Control)

The **JVM controls GC** entirely — you cannot directly trigger it, but you can **request** it using:

```
System.gc(); // Just a suggestion, not guaranteed
```

JVM decides **when and what to clean up** based on:

- Heap usage
- Object reachability
- GC algorithm (depends on JVM type and configuration)

□ GC Process (Simplified)

1. **Mark phase** → JVM marks all reachable objects (those still referenced).
 2. **Sweep phase** → JVM deletes unreferenced (garbage) objects.
 3. **Compact phase** (in some algorithms) → Rearranges objects to reduce fragmentation.
-

□ Types of Object References in Java

Java defines multiple reference types to control **how the GC treats objects**. All are defined in the `java.lang.ref` package.

1. Strong Reference (Default)

Most common type — any normal reference created using `new`.

```
String name = new String("Suraj");
```

◆ GC Behavior:

- **Never collected** as long as a strong reference exists.
- Causes **OutOfMemoryError** if memory runs out and objects are still strongly referenced.

✓ Use Case:

- Regular object references (default in Java).
-

2. Soft Reference

Created using `SoftReference` class.

```
SoftReference<String> softRef = new SoftReference<>(new String("SoftData"));
```

◆ GC Behavior:

- Collected **only if JVM is low on memory**.
- Ideal for **caching** — JVM keeps the object until memory is needed elsewhere.

✓ **Use Case:**

- Implementing **in-memory caches** (e.g., image cache, query cache).
-

3. Weak Reference

Created using `WeakReference` class.

```
WeakReference<String> weakRef = new WeakReference<>(new String("WeakData"));
```

◆ **GC Behavior:**

- Collected **as soon as no strong/soft references exist**.
- GC removes them eagerly.

✓ **Use Case:**

- **WeakHashMap** — automatically removes entries when keys are no longer in use.
-

4. Phantom Reference

Created using `PhantomReference` and a `ReferenceQueue`.

```
PhantomReference<String> phantomRef =  
    new PhantomReference<>(new String("PhantomData"), refQueue);
```

◆ **GC Behavior:**

- Collected **after finalization but before actual reclamation**.
- Used to perform actions **after object cleanup**.

✓ **Use Case:**

- **Post-cleanup tasks**, custom memory management frameworks.
-

□ Reference Type Comparison

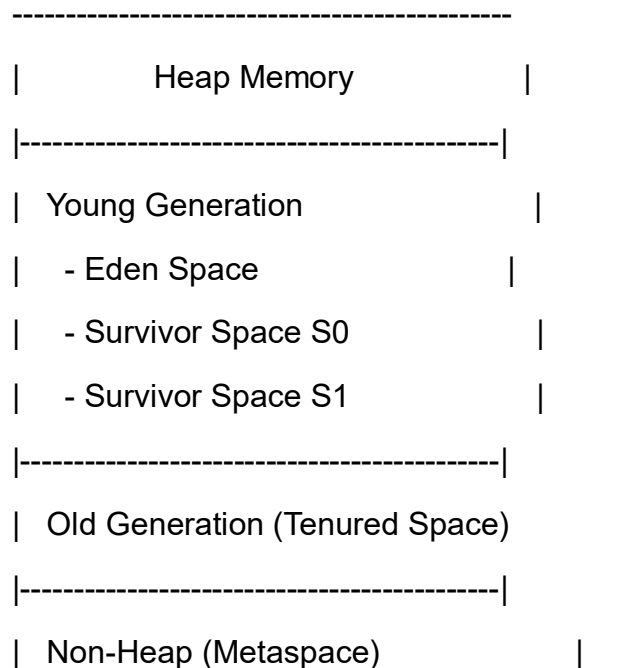
Reference Type	GC Priority (Collected When...)	Typical Use
Strong	Never (unless no reference exists)	Default object references
Soft	Low memory condition	Cache
Weak	Immediately if only weak refs remain	WeakHashMap, avoiding memory leaks
Phantom	After finalization, before memory reclaim	Cleanup tracking

Heap Memory Structure in Java (JVM Memory Model)

The **Heap** is the **runtime data area** from which memory for all **class instances (objects)** and **arrays** is allocated.

Java divides the **Heap memory** into several regions to efficiently manage the object lifecycle and garbage collection.

□ Heap Memory Layout



🔧 1. Young Generation

Newly created objects are allocated here.

It consists of:

- **Eden Space** → where new objects are first created.
- **Survivor Spaces (S0, S1)** → two smaller regions used to hold objects that survive garbage collections.

◆ Memory Flow in Young Gen:

1. When a new object is created → goes into **Eden space**.
2. When **Eden fills up** → a **Minor GC** is triggered.
3. Live (reachable) objects are **moved from Eden** → **Survivor S0**.
4. Next GC cycle:
 - Live objects in **Eden and S0** move → **S1**.
 - Then **S0 and S1 swap roles** (one always empty).
5. Each time an object **survives a GC**, its **age counter** increases.
6. When the **age reaches a threshold (e.g., 15 by default)** → it gets **promoted to Old Generation**.

□ 2. Old Generation (Tenured Space)

- Stores **long-lived objects** (those that survived multiple Minor GCs).
- Usually larger than the Young Gen.
- **Major GC (Full GC)** cleans this space.
- Objects here stay longer, so GC is less frequent but more expensive.

⚙️ 3. Non-Heap Memory (Metaspace)

- Stores **class metadata, method info, and static variables**.

- Replaced **PermGen** in Java 8.
 - Grows dynamically — not part of heap space.
 - Memory comes from **native OS memory**.
-

🔄 Object Lifecycle (Movement Between Regions)

Step	Memory Area	Description
1	Eden Space	Object created
2	Survivor S0 / S1	Survived 1 or more Minor GCs
3	Old Generation	Promoted after reaching age threshold
4	GC Deletes	Removed when no longer reachable

◆ Promotion Threshold

- Defined by JVM parameter:
- `-XX:MaxTenuringThreshold=<n>`

Default = **15**

- If an object survives **n** GC cycles → moves to **Old Gen**.
-

☐ Garbage Collection Types

1. Minor GC

- Cleans **Young Generation (Eden + Survivor spaces)**.
 - Happens frequently and quickly.
 - When **Eden** is full:
 - Live objects copied to **S0/S1**.
 - Unreferenced objects removed.
 - Triggers **promotion** to Old Gen if age threshold is reached.
-

2. Major GC (Old GC / Full GC)

- Cleans **Old Generation**.
 - Much **slower** than Minor GC — may **pause application threads**.
 - Triggered when:
 - Old Gen is full.
 - System explicitly calls `System.gc()`.
-

🔍 Mark and Sweep Algorithm (Used in GC)

The **core logic** behind garbage collection:

□ Steps:

1. **Mark phase** → Identify (mark) all reachable objects from GC roots (e.g., stack variables, static references).
2. **Sweep phase** → Remove (sweep) all unmarked (unreachable) objects, freeing memory.
3. **Compact phase (optional)** → Rearranges memory to eliminate fragmentation.

Garbage Collector Versions in Java (in short)

GC Type	Description	Key Features	Best For
1. Serial GC (-XX:+UseSerialGC)	Oldest, simplest GC — single-threaded.	• Works on one thread. • “Stop-the-world” pauses. • Uses Mark–Sweep–Compact.	Small applications, single-core systems.
2. Parallel GC (-XX:+UseParallelGC)	Multi-threaded version of Serial GC.	• Multiple threads for GC. • High throughput.	Multi-core CPUs, batch jobs. (Default in Java 8)

GC Type	Description	Key Features	Best For
3. CMS (Concurrent Mark Sweep) (-XX:+UseConcMarkSweepGC)	Reduces pause time by collecting alongside app threads.	• Still has “stop-the-world”. • Concurrent marking & sweeping. • Less pause, but more CPU usage. • No compaction → fragmentation possible.	Low-latency apps (e.g., web servers). <i>(Deprecated after Java 14)</i>
4. G1 GC (Garbage First) (-XX:+UseG1GC)	Modern default GC since Java 9+.	• Heap divided into regions. • Parallel + concurrent. • Predictable short pauses. • Compacts automatically.	Large heaps, real-time or low-latency systems. <i>(Default in Java 11+)</i>

Quick Summary Mnemonic:

- 👉 Serial → Simple
- 👉 Parallel → Fast (multi-threaded)
- 👉 CMS → Low pause, fragmented
- 👉 G1 → Balanced & modern

You are facing a memory-related issue in a Java project — how do you identify the code causing it?

✅ 1 Detect Symptoms

First, recognize the type of memory issue you’re dealing with:

- **High memory usage**

- **Frequent Full GCs**
- **OutOfMemoryError**
- **App slowness / GC pauses**
- **Heap exhaustion (Old Gen full)**

Check application logs or monitoring dashboards for:

- java.lang.OutOfMemoryError: Java heap space
- java.lang.OutOfMemoryError: Metaspace
- java.lang.OutOfMemoryError: GC overhead limit exceeded

✓ **2 Enable GC & Memory Logging**

Start with JVM GC logging to understand what's happening:

-verbose:gc

-XX:+PrintGCDetails

-XX:+PrintGCDateStamps

-Xlog:gc*,gc+heap=debug:file=gc.log:time,uptime,level,tags

Analyze the GC log:

- Is **Full GC** happening too often?
- Is **heap size** always close to the limit?
- Is **Old Gen** not freeing up space?

If Old Gen fills frequently → likely **memory leak or excessive object retention**.

✓ **3 Capture a Heap Dump**

Take a **heap dump** when the issue occurs (or just before OutOfMemoryError):

◆ **Option 1: Using JVM flags**

-XX:+HeapDumpOnOutOfMemoryError

-XX:HeapDumpPath=/path/to/dump.hprof

◆ **Option 2: Manually**

Use tools:

```
jmap -dump:format=b,file=heapdump.hprof <pid>
```

💡 Heap dump is a snapshot of everything in memory at that moment — invaluable for root cause analysis.

✓ 4 Analyze the Heap Dump

Use one of these tools:

- **Eclipse MAT (Memory Analyzer Tool)**
- **VisualVM**
- **JProfiler / YourKit**
- **Eclipse / IntelliJ built-in analyzers**

What to look for:

- **Dominant object types** (what is consuming memory?)
 - **Leak suspects** (Eclipse MAT shows “Leak Suspects Report”)
 - **Large collections** that never shrink (e.g., HashMap, List, ConcurrentHashMap)
 - **Static variables** holding references to unused objects
 - **Thread locals** retaining memory
 - **ClassLoaders** retaining references in web apps
-

✓ 5 Reproduce & Narrow Down in Dev

Once you know which object types or classes consume memory:

- Add **profiling or metrics** around those code paths.
 - Check for **caches not cleared**, **collections not cleaned**, or **listeners not removed**.
 - Run load test / stress test and monitor memory using **VisualVM** or **JProfiler**.
-

✓ 6 Common Code Patterns That Cause Memory Leaks

Pattern	Example	Why it leaks
Static collections	<code>static List<User> users = new ArrayList<>();</code>	Objects never eligible for GC
Unclosed resources	Streams, Sockets, DB connections	References remain until closed
Listeners / Callbacks	Swing, event handlers not removed	Still referenced
ThreadLocal misuse	ThreadLocal without <code>remove()</code>	Retained by thread
Poor cache logic	Manual maps without eviction	Memory grows indefinitely
ClassLoader leaks	Redeploying web apps in Tomcat	Old classes not GC'd

🔍 Monitor at Runtime

Use **JVM monitoring tools** to track heap usage in real-time:

- **VisualVM**
- **Java Mission Control (JMC)**
- **JConsole**
- **Prometheus + Grafana** dashboards
- **jstat** command:
- `jstat -gcutil <pid> 5s`

→ shows heap usage %, GC counts, and pause behavior.

🔧 Verify the Fix

After code changes:

- Re-run with same load.
- Confirm reduced memory footprint.
- Check GC frequency and heap occupancy.

- Capture heap dump again → verify reduced retention.
-

□ Sample Interview Answer (Concise Version)

“If we get memory-related issues like high heap usage or OutOfMemoryError, I first enable GC logs and monitor heap trends using tools like VisualVM or JConsole.

If the Old Gen is consistently full, I capture a heap dump using jmap or JVM's -XX:+HeapDumpOnOutOfMemoryError flag.

Then I analyze it with Eclipse MAT to find which classes hold the most memory or if any static collections are retaining objects.

Once I find the culprit (like an unbounded cache or unclosed resource), I fix the code, re-run under load, and verify GC behavior and heap stability.”